

Dai Sistemi Sequenziali al Parallelismo

Nei processori tradizionali non-pipelined (come il Motorola 68k), l'architettura era divisa in una parte operativa e una di controllo.

- **Esecuzione sequenziale:** L'intero hardware era dedicato a una singola istruzione alla volta. Nella fase di *fetch* si usava il Program Counter, nella fase di esecuzione si usava l'ALU, e così via.
- **Limite strutturale:** L'unico modo per aumentare le prestazioni in un sistema del genere era incrementare la frequenza di clock, un approccio che incontra rapidi limiti fisici (calore, consumi).

Per superare questo limite e aumentare la velocità, l'architettura dei calcolatori ha intrapreso due strade, non mutuamente esclusive:

1. **Aumentare il numero di unità elaborative** (Più nodi).
2. **Rendere l'unità elaborativa più performante** (Parallelismo di nodo, ovvero il Pipelining).

Tassonomia delle Architetture Parallele (Livello di Sistema)

Quando si aumentano le unità elaborative, si entra nel dominio delle architetture parallele (Tassonomia di Flynn). Le due categorie principali trattate sono:

- **SIMD (Single Instruction, Multiple Data):** * Una singola istruzione viene eseguita contemporaneamente su più set di dati.
 - *Applicazioni:* Ideale per Signal Processing, elaborazione grafica e Intelligenza Artificiale, dove si fanno pesanti operazioni matriciali (es. sommatoria di prodotti).
 - *Caratteristica:* Non prevede istruzioni di salto (branch) per i singoli dati, si concentra su operazioni aritmetiche parallele.
- **MIMD (Multiple Instruction, Multiple Data):**
 - Più processori eseguono istruzioni diverse su dati diversi. Questa categoria si divide in due architetture principali:
 - **Multicomputer (Memoria Distribuita):** Sistemi formati da computer indipendenti, ognuno con la propria memoria e periferiche, collegati da una rete di interconnessione. Comunicano tramite operazioni di I/O (scambio di messaggi).
 - **Multiprocessore (Memoria Condivisa):** Più processori comunicano attraverso uno spazio di memoria condiviso, tipicamente collegati su un bus comune.

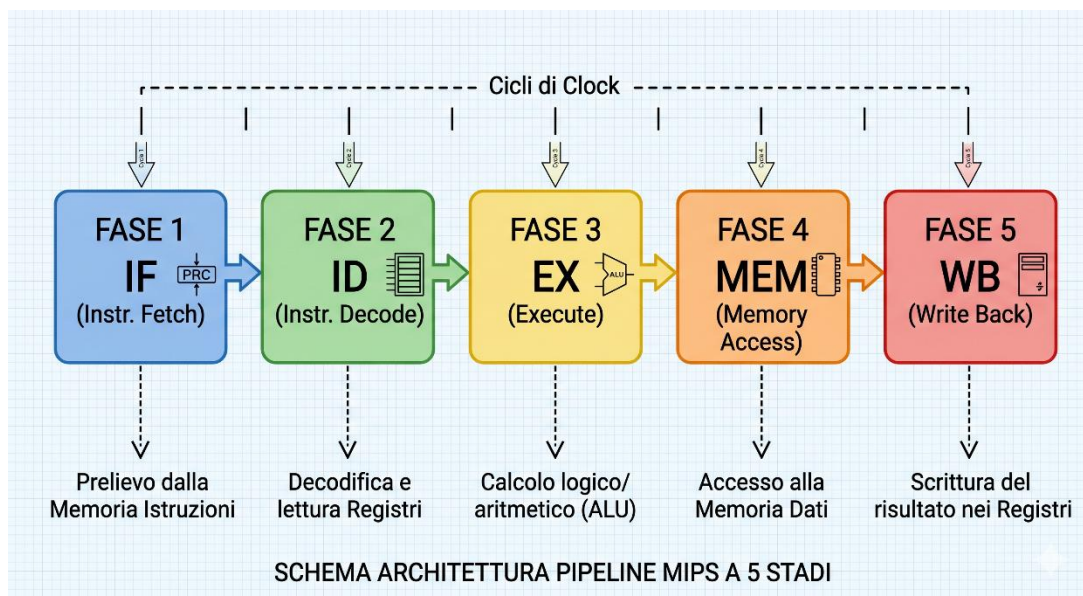
Nota di design: È possibile creare architetture ibride ad altissime prestazioni, ad esempio un Multicomputer i cui nodi sono a loro volta sistemi Multiprocessore (cluster).

Architettura RISC e Pipelining (Il Modello MIPS)

Per implementare il *parallelismo di nodo*, i sistemi moderni si rifanno all'architettura **RISC** (Reduced Instruction Set Computer) e al modello del processore **MIPS** teorizzato da Patterson.

Invece di usare una complessa logica microprogrammata, si utilizza una **logica cablata**, molto più veloce, dove ogni fase dell'istruzione ha la sua unità di controllo dedicata.

La Pipeline a 5 Stadi



Un'istruzione viene divisa in 5 fasi che impiegano lo stesso tempo (tempo di ciclo Δt). Le fasi lavorano in *pipeline*: mentre un'istruzione è in esecuzione, quella successiva viene decodificata e una terza viene prelevata.

□ **IF (Instruction Fetch - Prelievo Istruzione):** L'unità di controllo usa il Program Counter (PC) per sapere dove si trova la prossima istruzione. Va nella **Memoria Istruzioni**, preleva l'istruzione a 32 bit e, contemporaneamente, un sommatore dedicato incrementa il PC per puntare all'istruzione successiva.

□ **ID (Instruction Decode - Decodifica e Lettura Registri):** L'hardware "smonta" l'istruzione a 32 bit per capire quale operazione deve essere fatta (OpCode). Contemporaneamente, legge i valori contenuti nei **Registri** che serviranno per l'operazione. È qui che il sistema si accorge se si tratta di un'operazione aritmetica, di un accesso a memoria o di un salto (Branch).

□ **EX (Execute - Esecuzione):** L'**ALU** (Arithmetic Logic Unit) entra in azione. Se è un'operazione matematica (es. somma), fa il calcolo. Se è un'istruzione di caricamento/salvataggio in memoria (Load/Store), l'ALU calcola l'indirizzo fisico di memoria in cui andare.

□ **MEM (Memory Access - Accesso a Memoria):** Questa fase è attiva solo per le istruzioni Load/Store. Il processore accede alla **Memoria Dati** (o Cache L1 Dati) per leggere un dato (Load) o scriverci un valore (Store). Per le normali operazioni

matematiche (es. $R1 = R2 + R3$), questo blocco non fa nulla e passa semplicemente il risultato al blocco successivo.

□ **WB (Write Back - Scrittura a Ritroso):** Il risultato finale dell'operazione (che provenga dall'ALU nella fase EX o dalla Memoria Dati nella fase MEM) viene scritto definitivamente nel **Registro** di destinazione specificato, rendendolo disponibile per le istruzioni future.

I Vincoli dell'Architettura RISC

Per far sì che la pipeline non si blocchi, il set di istruzioni impone vincoli rigidi:

- **Lunghezza fissa (32 bit):** Tutte le istruzioni hanno la stessa dimensione. Se fossero di lunghezza variabile, il fetch richiederebbe tempi imprevedibili (rompendo la sincronia della pipeline).
- **Architettura Load/Store:** Le operazioni aritmetico-logiche (ALU) si fanno **solo** sui registri o con operandi immediati. È vietato l'indirizzamento diretto in memoria per l'ALU (altrimenti la fase EX richiederebbe un accesso in memoria, accavallandosi con la fase MEM).
- **3 Operandi:** Le istruzioni usano registri generali nel formato $R1 = R2 + R3$.
- **Caricamento di indirizzi a 32 bit:** Poiché un'istruzione è a 32 bit e deve contenere sia il codice operativo (OpCode) che il registro di destinazione, non c'è spazio per un indirizzo a 32 bit completo. Servono **due istruzioni** per caricare la parte alta (Upper) e la parte bassa (Lower) dell'indirizzo nel registro.

non puoi
fare Addi
#20,\$8000



Se la fase IF preleva un'istruzione dalla memoria e contemporaneamente la fase MEM preleva un dato per un'altra istruzione, un'unica memoria creerebbe un collo di bottiglia strutturale.

- **Soluzione:** I sistemi pipelined utilizzano memorie fisicamente o logicamente separate per Istruzioni e Dati.
- **Gerarchia:** Queste memorie non possono essere lente. Si usano **Cache Primarie (L1)** sul processore (divise in L1 Dati e L1 Istruzioni). Se c'è un *miss*, si passa alla **Cache Secondaria (L2)** esterna, e solo in caso di ulteriore miss si va in Memoria Principale (RAM).

Problematiche della Pipeline e Soluzioni

La sovrapposizione delle istruzioni genera dei conflitti che il processore deve risolvere.

A. Conflitti di Controllo (Il problema dei Salti)

Le istruzioni di salto (Branch) costituiscono circa il 25% del codice. Il problema è che il processore scopre se deve saltare o meno solo nella fase di decodifica (ID) o esecuzione (EX). Nel frattempo, ha già caricato le istruzioni successive sbagliate.

- **Conseguenza:** Il processore deve "svuotare" la pipeline, buttando via il lavoro fatto e perdendo cicli di clock.

B. Conflitti sui Dati (RAW, WAR)

Avvengono quando un'istruzione ha bisogno di un dato calcolato dall'istruzione immediatamente precedente, ma non ancora salvato nei registri.

- *Esempio:* 1. $R1 = R4 + R3$ ($R1$ viene scritto nel registro solo nella fase 5 - WB)
2. $R1 = R1 + R2$ ($R1$ serve già nella fase 3 - EX)
- *Soluzione:* **Forwarding (o Cortocircuito).** Si utilizza un multiplexer prima dell'ALU nella fase 3. Il processore "vede" tutte le istruzioni in volo e, se nota una dipendenza, abilita il multiplexer per prelevare il risultato appena uscito dall'ALU dell'istruzione precedente, senza aspettare che arrivi alla fase 5.

C. Conflitti Strutturali

Avvengono quando due fasi richiedono la stessa risorsa hardware.

- *Esempio dell'incremento del PC:* Non è possibile usare l'ALU per incrementare il Program Counter durante la fase IF, perché l'ALU potrebbe essere occupata dalla fase EX di un'altra istruzione. Serve un sommatore hardware dedicato unicamente all'incremento del PC. Ogni blocco non deve rompere le scatole agli altri blocchi.

D. Gestione delle Interruzioni (ISR - Interrupt Service Routine)

In una pipeline con 5 istruzioni in volo, capire chi ha generato l'eccezione è complesso:

- **Interruzioni Esterne (Asincrone):** Sono facili da gestire. Derivano dall'I/O, si accodano alla fine del ciclo corrente senza interrompere brutalmente la pipeline in mezzo a un'istruzione.
- **Eccezioni Interne (Sincrone):** Sono problematiche perché possono avvenire fuori ordine. Un errore critico nella fase IF (es. accesso illegale) interrompe subito il sistema, mentre una divisione per zero generata da un'istruzione precedente verrà rilevata solo più tardi, nella fase EX. I processori moderni gestiscono queste problematiche localmente in ogni stadio per mantenere lo stato preciso della macchina (Precise Exceptions).

Oltre la Singola Pipeline: Architetture Superscalari

Cosa succede quando vogliamo spingerci oltre il limite di 1 istruzione per ciclo di clock ($IPC = 1$) di una singola pipeline MIPS?

- **Architettura Superscalare:** Molti processori moderni (come quelli Intel o ARM) contengono *più* pipeline che operano in parallelo. Dispongono di più unità ALU, più unità di caricamento dati, ecc. Il processore deve prelevare più istruzioni contemporaneamente, analizzarne le dipendenze in tempo reale (hardware di coordinamento molto complesso) ed emetterle ("issue") verso le diverse pipeline in parallelo, permettendo al sistema di completare 2, 3 o 4 istruzioni per ogni colpo di clock.

Conflitti nei sistemi basati su pipeling- La gestione dei salti

In presenza di un salto non deve essere sempre prelevata dalla memoria l'istruzione successiva, e può non essere facile o possibile determinare subito l'istruzione a cui saltare.

Quando il processore preleva una istruzione, non sa che tipo di istruzione ha prelevato finché non la interpreta (fase ID), ma, a questo punto, ne avrà già presa un'altra (quella immediatamente successiva). Potrebbe rendersi conto che l'istruzione precedente era un salto, dovendo quindi saltare ad un'istruzione diversa da quella successiva, rendendo, quindi, il successivo prelievo inutile.

L'importante, in questa situazione, sarebbe, rendersi conto che l'istruzione che sta evolvendo è un salto e che cosa segue questo salto, entro le prime due fasi o al massimo alla terza cioè nella fase Ex in quanto successivamente potremmo scrivere in memoria e nei registri, tenendo ormai traccia delle operazioni che non andavano fatte.

Se abbiamo un processore RISC, abbiamo probabilmente istruzioni di salto più semplici (tipo JNZ dove si va a verificare il solo flag Z), caratterizzate, quindi, da una condizione di salto semplice, in tal caso nella fase di decodifica è già possibile capire se il processore deve o meno effettuare il salto. Nel caso in cui invece la condizione di salto risulti essere più complessa allora la valutazione viene ad essere effettuata nella fase EX.

Considerando che almeno il 25% delle istruzioni sono salti, pensare di buttare molte istruzioni quando ci accorgiamo di un salto, risulterebbe molto inefficiente.

Consideriamo di avere

76 CMP R1, R3

80 BEQ 100

100 MOVE R1, R2

	(1)	(2)	(3)	(4)	(5)
80	IF	ID	EX	MEM	WB
84		IF	ID	EX	MEM
88			IF	ID	EX

→ tempo

L'istruzione 80 che sarebbe il salto quando attraversa le fasi non sappiamo ancora che è un salto, quindi verranno caricate 84 e 88. Solo quando l'istruzione 80 raggiunge la fase 3 lo scopriremo.

Se ce ne accorgiamo nella fase Ex, le altre due istruzioni non avranno fatto danni.

Nel caso in cui il salto non deve essere eseguito la pipe continua a funzionare normalmente e cioè se nel caso specifico la compare darà esito diverso da zero; se, però, il risultato è $\neq 0$ entra nel salto e deve essere eseguito, le istruzioni 84 e 88 dovranno essere eliminate (flush della pipe) e bisognerà prelevare l'istruzione 100 e successive.

Si crea un ritardo, che diminuisce la produttività della pipe, detto **branch penalty**.

- Approccio conservativo: nel momento in cui il processore interpreta una istruzione come istruzione di salto (fase ID di decodifica dell'istruzione), ferma la pipe, disabilita la propagazione della istruzione che era stata erroneamente già prelevata, determina l'istruzione a cui saltare (fase EX) e la preleva.

	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
80	IF	ID	EX	MEM	WB			
84		IF						
88								
100				IF	ID	EX	MEM	WB
104					IF	ID	EX	MEM
108						IF	ID	EX
112							IF	ID

→ tempo

Ovviamente il prezzo da pagare per il blocco di una pipe è tanto maggiore quanto più lunga è la pipe. Inoltre se la percentuale di salti nel programma è significativa (es. 25%) si ha un notevole sottoutilizzo delle potenzialità della pipe, che viene continuamente bloccata.

Molti ritardi possono essere evitati dal programmatore o dal compilatore: il programmatore può contribuire al buon funzionamento del sistema, scrivendo le istruzioni in un ordine tale da minimizzare le probabilità di stallo della pipe. Nel caso di un costrutto if-then-else, ad esempio, conviene inserire nel ramo then l'alternativa più probabile perché il ramo then corrisponde all'istruzione presa in modo sequenziale e non all'if e quindi al salto. Il compilatore, da parte sua, può operare vari accorgimenti; ad esempio, siccome un ciclo for è sempre tradotto in un if-then-else, esso deve inserire l'alternativa più probabile nel ramo then. Nel caso di due cicli innestati, uno di 10 iterazioni e uno di 1000, è conveniente inserire il più lungo all'interno, in tal modo si verifica un errore di caricamento pipe ogni 1000 iterazioni. Il compilatore può capire se il programmatore ha posizionato i due cicli in un ordine non appropriato e invertirli, nel casi in cui ciò sia algebricamente possibile.

- Approccio Ottimistico. (Branch Prediction) E' quello più complicato e maggiormente diffuso. In questo caso tentiamo di fare una previsione su quale sia il ramo da eseguire in una istruzione condizionale.

```

100  if condizione
      then
104  istruzione
      else

```

112 istruzione

Il ramo then segue immediatamente l'if, mentre l'else si trova a 112 e quindi l'esecuzione richiede un salto. BNE L1

Quello che si tenta di fare è indovinare quale ramo debba essere selezionato, ovvero capire se l'istruzione seguente si trova a 104 oppure 112. Generalmente è impossibile saperlo a priori e allora si fa un'analisi predittiva.

La prima volta che eseguo non sappiamo come si comporterà e allora usiamo una strategia conservativa ovvero preleviamo l'istruzione successiva. Se la decisione è sbagliata e cioè alla fine abbiamo saltato significa che le istruzioni erano state cancellate e poi abbiamo dovuto fare il reset dei registri delle fasi precedenti e caricare l'istruzione corretta.

Quando in seguito si ripresenta l'istruzione 100 l'obiettivo è non sbagliare di nuovo.

Allora si costruisce una Branch Prediction Table ovvero una tabella associativa (HASH) in cui abbiamo delle chiavi e dei valori. La chiave è l'istruzione e il valore è l'istruzione successiva. Quando il processore sbaglia riempie questa tabella in modo che la volta successiva la consulta e preleva l'istruzione che prima mi avrebbe fatto fare la scelta corretta. Se la chiave non è presente nella tabella allora si opera normalmente prendendo l'istruzione successiva. Questo può significare due cose: la prima è che se non è presente allora potrebbe essere la prima volta che incontro quell'istruzione e quindi entra in gioco il metodo conservativo, la seconda è che abbiamo già incontrato l'istruzione, ma non abbiamo sbagliato e quindi continueremo a fare bene (non è detto ovviamente, se la condizione cambia sbagliamo)

Questa tabella si trova nella fase 1.

Il problema nasce in presenza di cicli innestati:

for i...	92	CMP
		JMP
for j...		CMP
	100	JXX
	104	...
		...
	112	JMP 92

la prima volta che esegue la riga 96 non succede niente, quindi carica le successive. Entra nel ciclo interno e arriva all'istruzione 100. Nella tabella non trova niente perché è la prima volta che la esegue. Allora carica sicuramente l'istruzione 104 che verrà eseguita poiché è la prima iterazione del ciclo interno quindi la condizione non è falsa e non

scriverà neanche niente per la tabella.

Continua così per n-1 iterazioni interne. Quando arriva all'ultima la condizione è falsa e prevede un salto a 112. Ma lui caricava sempre 104. Quindi ora ha sbagliato e allora se lo segna nella tabella.

Istruzione 100, next 112.

A 112 ritorna al ciclo più esterno

Quello che succede però che appena ricomincia il ciclo interno, lui si era segnato che dopo l'istruzione 100 doveva andare all'istruzione 112. Quindi succede che

carica la 112 e poi si accorge di aver sbagliato, quindi cancellare e mettere quella giusta.

Dovrebbe essere in grado di capire che deve andare a 112 solo quando è l'ultima iterazione del ciclo interno.

I processori allora evolvono tra 4 stati codificati su due bit: NON SALTARE (FORTE|00), NON SALTARE (DEBOLE|01), SALTA (FORTE|10), SALTA (DEBOLE|11).

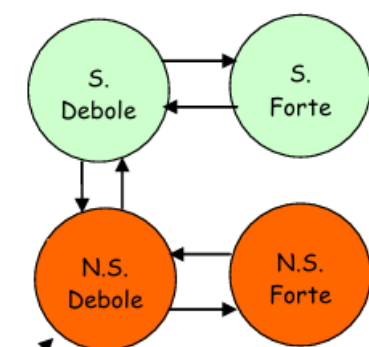


Figura 9: I 4 stadi del Branch Prediction.

Il processore lavora sempre nello stato non salta forte, ovvero il processore è sicurissimo che la condizione sarà falsa ovvero l'istruzione è la successiva, fin quando c'è un primo errore. Quando c'è un primo errore significa che deve saltare e che quindi la condizione è vera. Memorizza la coppia di indirizzi nella tabella ed entra nello stato "non salta debole", ovvero inizia a preoccuparsi, ma non dice subito ok la volta dopo devo saltare, ma indebolisce semplicemente il non saltare. La prima volta è forte come concetto poiché usiamo un metodo conservativo.

La volta successiva che ricade in quell'istruzione, non effettua il salto (come facevamo prima nel modello base). Ma controlla cosa accade. Aveva segnato che lo stato era 01. Se la condizione è di nuovo falsa quindi l'istruzione è la seguente allora ritorna nello stato 00.

Se invece la condizione è vera per la seconda volta, significa che ho sbagliato due volte, perché due volte ho caricato l'istruzione successiva e invece non lo era perché ho saltato.

Avendo sbagliato due volte cambia stato e va in 10 quindi debolmente saltare. Il che può significare "devo saltare ma non sono del tutto convinto".

Se ricapitiamo di nuovo sull'istruzione il processore va nella tabella associativa guarda 10 come stato e allora prende l'istruzione indicata come next.

Se la condizione ancora una volta è vera e quindi salta passa nello stato 11 e si convince del fatto che quando capita quell'istruzione, la condizione è vera e deve saltare.

Se la condizione è falsa invece subisce il ritardo perché aveva messo l'istruzione in cui saltavamo e invece era la successiva. Da 10 torna in 10.

Quindi se ci troviamo in 00 oppure 11 sono gli stati forti. Un singolo errore non fa cambiare la predizione, ma gli fa perdere solo fiducia. Quindi si può dire che ha 2 errori prima di cambiare idea.

Riprendiamo l'esempio di prima. Nel ciclo interno per $n-1$ volte non sbaglia quindi lo stato è 00

Alla n esima volta il ciclo è finito e deve tornare sopra quindi ha sbagliato una volta. Scrive nella tabella associativa

100 112 stato=NSD (non saltare debole)

Eseguiamo per la seconda volta il ciclo interno. Arriva all'istruzione 100 e comunque carica 104 che sarà la scelta giusta, quindi ritorna nello stato NSF

Si noti dunque che il processore sbaglia alla fine della prima esecuzione e alla fine della seconda. Supponendo allora che il ciclo esterno sia di 10 iterazioni e quello interno di 1000, il processore sbaglia una volta per ciascuna esecuzione del ciclo interno (sull'ultima iterazione, un errore inevitabile), più una volta sul ciclo esterno, e quindi $10 + 1 = 11$ volte soltanto, su $10 \times 1000 = 10000$ iterazioni.

Da questo si capisce anche il perché il numero di cicli esterni deve essere minore del numero di cicli interni, in quanto come visto dal conteggio degli errori nella pipe, se ho più cicli esterni commetto più errori rispetto al caso opposto.

Trattazione secondo Patterson e Tanenbaum

Utilizzano un'automa con transizioni dirette tra predizioni opposte in caso di due errori e si concentrano esclusivamente sulla **logica di predizione** (i 2 bit di stato).

Non fanno alcun riferimento all'implementazione di una branch table, ma solo ai due bit di riferimento attraverso cui gestire i salti. L'automa visto in precedenza e la branch table fanno affidamento ad un processore reale. Mentre le due trattazioni di Patterson e Tanenbaum fanno una trattazione didattica.

- *Patterson (architettura DLX)*: Ipotizza che la condizione e l'indirizzo di salto siano risolti già nella fase di decodifica (ID).
- *Tanenbaum*: Ipotizza che l'indirizzo di destinazione sia già magicamente a disposizione (simile a un salto incondizionato).

In un processore reale non basta "indovinare" se il salto avverrà o meno (i 2 bit). Bisogna anche sapere **dove** saltare prima ancora di aver decodificato l'istruzione. Per questo la logica a 2 bit deve essere integrata all'interno di una **Branch Target Buffer (BTB)**, una tabella di memoria associativa che salva gli indirizzi target dei salti calcolati in precedenza.

Lo stadio IF ad ogni singolo colpo di clock deve prelevare un'istruzione e **aggiornare immediatamente il Program Counter (PC)** per sapere quale istruzione prelevare al colpo di clock successivo.

Se l'istruzione appena prelevata è un salto (branch), lo stadio IF non lo sa ancora! Lo scoprirà solo nello stadio successivo (Decode). Ma nel frattempo, il clock fa un tic, e l'IF *deve* prelevare l'istruzione successiva. Quale prende? Di default prende quella successiva in memoria ($PC + 4$).

Se ci affidassimo solo ai 2 bit di predizione, e ad esempio dice che dobbiamo saltare, **dove** saltiamo? L'indirizzo di destinazione (es. l'inizio di un ciclo for) deve ancora essere calcolato, è un'operazione che richiede tempo e hardware (un sommatore).

Il risultato senza BTB: Anche se hai predetto correttamente *che* salterai, devi comunque fermare la pipeline (inserire bolle/stalli) per aspettare che l'indirizzo venga calcolato. Hai vanificato l'utilità della predizione.

Il BTB è una memoria hardware velocissima integrata nello stadio IF. Ad ogni colpo di clock, mentre l'IF legge la memoria istruzioni usando il PC, usa quello stesso PC per interrogare il BTB. Se c'è un "hit" (il PC è nel BTB), il BTB restituisce **due cose contemporaneamente**:

1. I 2 bit di predizione (se saltare o no).
2. **L'indirizzo di destinazione esatto** (calcolato e salvato in un'esecuzione precedente).

In questo modo, se la predizione è "Salta", lo stadio IF ha già l'indirizzo pronto e può aggiornare il PC in quello stesso ciclo di clock. Nessun ritardo, nessuna bolla.

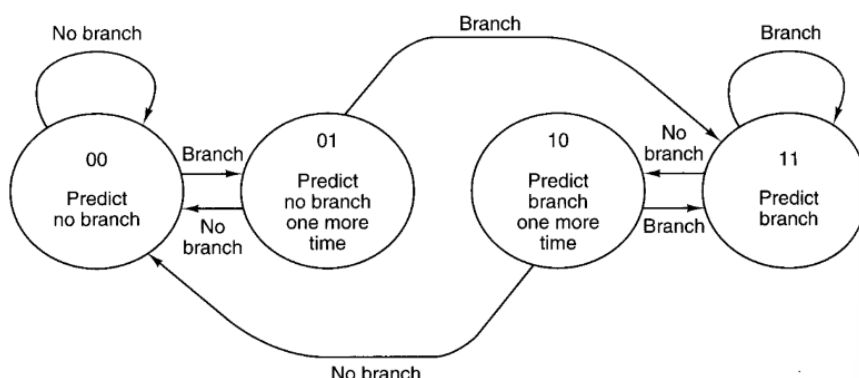
La semplificazione di affidarsi alla fase di decodifica **stravolge lo scopo del Decode**: Lo stadio ID dovrebbe solo "capire" quale istruzione è arrivata e leggere i registri. Se gli chiedi di risolvere un salto, devi inserire dentro lo stadio ID un Sommatore dedicato (per calcolare PC + offset) e un Comparatore (per verificare ad esempio se $R1 == R2$). Questo significa duplicare hardware costoso che si trova già nello stadio ALU (Execute).

Peggiora drasticamente i Data Hazards (Dipendenze sui dati): Questo è il problema più critico.

ADD R1, R2, R3 // Scrive il risultato in R1 (nello stadio EX o WB)
BEQ R1, R0, loop // Salta se R1 è uguale a R0

Se l'istruzione BEQ deve valutare la condizione nello stadio **Decode**, ha bisogno del valore aggiornato di R1 prestissimo! Ma l'istruzione ADD precedente sta appena calcolando R1 nello stadio Execute. Non c'è tempo fisico per fare il forwarding (cortocircuitare il dato) all'indietro così in fretta. Saresti costretto a inserire stalli (fermare il processore) praticamente ogni volta che un salto dipende da un calcolo fatto poco prima.

In entrambi i casi concordano sull'utilizzo del seguente automa:



Conflitti nei sistemi basati su pipelining- La gestione del conflitto sui dati

Se vogliamo andare veloce una delle soluzioni sono i sistemi pipelined. Uno dei problemi sono i salti. Possiamo risolvere con tecniche di prevenzione.

Il problema principale è che noi quando facciamo delle operazioni dobbiamo avere delle memorie veloci. Queste però hanno capacità limitate.

Se due istruzioni consecutive tentano di accedere contemporaneamente ad uno stesso registro/locazione di memoria, si generano conflitti sui dati, classificati secondo due tipologie:

- conflitti *Read after Write* (R/W);
- conflitti *Write after Read* (W/R).

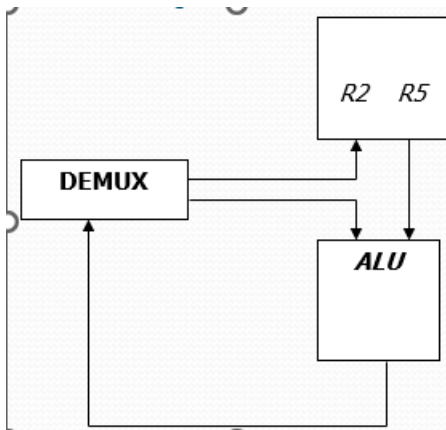
i $R2 = R1 + R3$

i+1 $R4 = R2 + R5$

	(1)	(2)	(3)	(4)	(5)
i	IF	ID	EX	MEM	WB
i+1		IF	ID	ID	EX

quando l'istruzione i+1 arriva alla fase ID, necessita del valore calcolato nella fase precedente, che però ancora non è stato memorizzato nel registro R2, perché si trova nella fase EX.

Per risolvere questo conflitto si utilizza la *tecnica dell'anticipo degli operandi (Operand Forwarding)*, cioè si crea un canale (hardware), che renda i risultati disponibili appena l'ALU li calcola.



Questa tecnica è realizzabile solo se si opera su registri del processore ed ogni fase impiega un solo ciclo di clock (processori RISC). È una tecnica hardware essenziale per minimizzare gli stalli (le cosiddette "bolle") nella pipeline causati dai **data hazard** di tipo RAW (Read After Write).

Il dato calcolato dall'ALU viene inviato dal demultiplexer contemporaneamente ai registri R del processore, ma anche alla stessa ALU, così da renderlo subito disponibile per il passo successivo. La tecnica dell'anticipo degli operandi consiste nel creare un canale (hardware), che renda i risultati disponibili appena l'ALU li calcola. In questo modo durante il 4° ciclo di clock il nuovo valore sarà memorizzato e, contemporaneamente, utilizzato dalla $i+1$, senza che la pipe entri in stallo.

Questa tecnica è realizzabile solo se si opera su registri del processore ed ogni fase impiega un solo ciclo di clock (processori RISC).

L'operand forwarding va per forza messo. È essenziale, senno non può fare neanche delle operazioni semplici.

Internal Forwarding

Nel caso di istruzioni, **in cui gli operandi sono locazioni di memoria** (processori CISC), l'approccio conservativo impone di fermare la pipe finché non si è sicuri che l'operazione concernente la locazione di memoria sia terminata (fase WB).

Inoltre, se troviamo una istruzione come l'istruzione i e il dato richiesto MEMA non risulta essere in cache, c'è un cache miss e la pipe comunque si ferma, quindi c'è uno stallo. In tal caso paghiamo molto, perché se la cache è 8-5 volte più veloce della RAM, allora probabilmente si blocca la pipe per tutti i quanti di tempo. La memoria è un dato molto critico in un sistema.

```
i      R1 = MEMA;  
i+1    R2 = R1 + R3;  
i+2    R4 = R2 + R5;  
i+3    R2 = R6 + R7;  
i+4    R4 = R2 + R4;
```

Utilizzando **le proprietà associativa e commutativa** è possibile evitare stalli della pipe cambiando l'ordine delle istruzioni.

La proprietà associativa consente di associare in un solo gruppo le istruzioni dipendenti tra loro;

La commutativa, invece, permette di individuare i gruppi di istruzioni invertibili, ovvero la cui inversione non causa errori di esecuzione. Tutto ciò è implementato da hardware posto prima della fase di EX. Se tali proprietà non valgono la pipe va in stallo. Per evitare stalli nel caso in cui le due proprietà descritte non siano praticabili, i processori implementano la tecnica dell'Internal Forwarding; la quale consiste nell'ibernare le istruzioni, che non possono essere eseguite subito, perché in conflitto con le precedenti, per iniziare ad eseguire le successive.



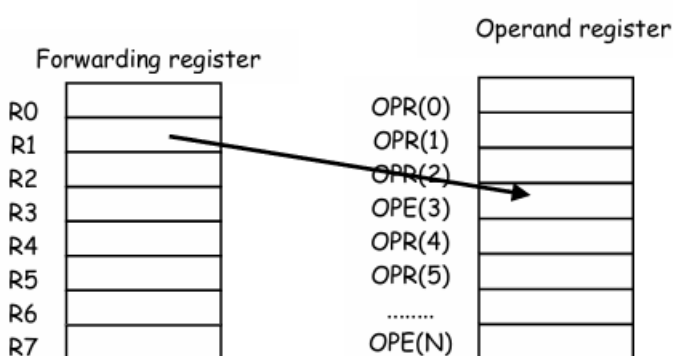
Per implementare questa tecnica si utilizza una memoria detta **tabella di ibernazione** statica ed interna al processore, in tale tabella annoto le istruzioni già decodificate quindi con gli operandi e l'operazione che deve essere effettuata. I registri dedicati all'ibernazione sono:

- 2 per l'addizione,
- 2 per la moltiplicazione,
- 1 per la divisione,
- 1 per l'operazione logica,
- 1 per il Load e
- 1 per lo Store;

	operando1		operando2		
	TAG	VALORE	TAG	VALORE	RISULTATO
ADD					
ADD					
MUL					
MUL					
DIV					
BOO					
LOAD					
STORE					

Se abbiamo, di un determinato tipo, più operazioni di quante è possibile ibernare nella tabella, non è possibile evitare il blocco della pipe, poiché il sistema è limitato.

E' possibile ibernare solo una operazione di Load e una di Store perché finché un'operazione in memoria non è conclusa non se ne possono attivare altre dello stesso tipo. Gli altri registri di ibernazione potrebbero essere aumentati per una maggiore efficienza ciò aumenterebbe il costo.



Nell'esempio il registro R2 è presente in due blocchi che abbiamo indicato eseguibili in maniera distinta dal processore in base alle due proprietà: commutativa e associativa; R2 dovrà assumere due valori diversi nell'ambito delle istruzioni ed alla fine devo trovare, per eseguire l'ultima istruzione, un registro R2

che deve contenere il valore corretto e cioè quello relativo all'ultima istruzione di assegnazione di R2.

Per fare questo bisogna capire che i registri del processore non contengono il valore effettivo, ma contengono un puntatore all'operando, per questo i registri del processore sono detti **Forwarding Register**, in base a questo fatto è possibile realizzare l'esecuzione non ordinata delle istruzioni. I registri del processore (Forwarding Register), quindi, sono utilizzati come puntatori ai Registri Operando; ad esempio, se R1 contiene il valore 3, ciò significa che il valore effettivo è memorizzato nel registro OPR(3).

Ad ogni registro operando è associato un contatore, per tener traccia del numero di operazioni sospese su quel registro ed un bit di 'occupato', che è pari a zero quando il dato contenuto nel registro è valido.

Esempio 1:

			valore	operazioni sospese	bit occupato
		OPR(0)	42		0
R0		OPR(1)	32		0
	8	OPR(2)			
R2	14	OPR(3)			
	15	...			
R4	16	OPR(7)			
R5	0	OPR(8)			
R6	1	OPR(N)			

All'inizio i registri si presentano in questo stato

Consideriamo l'istruzione i di caricamento di un dato dalla memoria, e supponiamo di non avere il dato nella cache (cache miss1), quindi non possiamo espletare l'operazione, questo significa che devo ibernare l'operazione di caricamento del dato dalla memoria nel registro R1.

Scriviamo nella tabella di ibernazione in corrispondenza dell'operazione di LOAD il tag cioè MEMA, quindi imporre nella colonna contenitore il valore 7, che rappresenta l'operand register dove troveremo il valore da caricare nel registro R1, quest'ultima informazione l'inseriamo anche nel registro R1.

	operando1		operando2		
	tag	valore	tag	valore	Contenitore
ADD					
ADD					
MUL					
MUL					
DIV					
BOO					
LOAD			MEMA		7
STORE					

nel registro R1 scriviamo 7, quindi è l'operando 7 e in corrispondenza di questo in operazioni sospese mettiamo 1.

Arriva quindi l'istruzione $i+1$, $R2=(R1+R3)$. L'hardware tenta di leggere R1 e R3. R3 è pronto. R1 invece contiene il Tag 7, che presenta una sospensione. Quindi dobbiamo rimandare anche questa istruzione, inseriamo anche questa istruzione che è un ADD nella tabella di ibernazione. Operando 1 è R1 e mettiamo 7, operando 2 è R3 e mettiamo come tag 14. Il contenitore in cui andrà è R2 e mettiamo 8.

			valore	operazioni sospese	bit occupato
		OPR(0)	42		0
R0	7	OPR(1)	32		0
	8	OPR(2)			
R2	14	OPR(3)			
	15	...			
R4	16	OPR(7)		2	
R5	0	OPR(8)		1	
R6	1	OPR(N)			

Quindi in R2 ci sarà 8 e cioè l'operando 8. Allora mettiamo operazioni sospese=1.

Inoltre R2 avendo bisogno di R1 fa aggiungere un'operazione sospesa anche ad R1, operando 7 che sale quindi a 2 operazioni sospese.

Consideriamo l'istruzione $i+2$, $R4=(R2+R5)$, dobbiamo ibernare anche questa istruzione non abbiamo ancora R2, conserviamo i dati nelle due tabelle e poniamo a 2 il numero di operazioni sospese su R2. In R4 scriveremo 15 quindi operando 15 e in R5 16 quindi operando 16.

	operando1		operando2		Contenitore
	tag	valore	tag	valore	
ADD	7		14		8
ADD	8		16		15
MUL					
MUL					
DIV					
BOO					
LOAD			MEMA		7
STORE					

			valore	operazioni sospese	bit occupato
		OPR(0)	42		0
R0	7	OPR(1)	32		0
	8	OPR(2)			
R2	14	OPR(3)			
	15	...			
R4	16	OPR(7)		2	
R5	0	OPR(8)		2	
R6	1	OPR(N)			

Quindi ci troviamo che R1 punta all'operando 7, R2 all'operando 8, R3 all'operando 14, R4 all'operando 15, R5 all'operando 16.

Ora abbiamo l'istruzione $i+3$ ($R2=R6+R7$), possiamo svolgere tale operazione, perché R6 e R7 puntano a zone i quali operandi non sono ibernati. Poniamo in R2 un nuovo puntatore (mettiamo 2 così punta ad operando 2), non perdiamo informazioni circa le operazioni precedentemente ibernate che coinvolgono R2, poiché lavoriamo sui puntatori e non sui valori effettivi, i puntatori cambiano, i valori effettivi restano. Svolgiamo l'operazione di addizione tra i valori puntati da R6 e R7.

			valore	operazioni sospese	bit occupato
		OPR(0)	42		0
R0	7	OPR(1)	32		0
	2	OPR(2)	74		0
R2	14	OPR(3)			
	15	...			
R4	16	OPR(7)		2	
R5	0	OPR(8)		2	
R6	1	OPR(N)			

R6 e R7 puntano a 0 e 1 in cui abbiamo i valori 42 e 32. Facciamo la somma e quindi in operando 2 ci troviamo il valore 74. Notiamo che impostiamo anche il bit 0 affianco per indicare che il valore è valido.

Ad un certo punto arriverà dalla memoria il dato atteso per caricare R1, quindi la serie di operazioni di caricamento delle istruzioni i+k viene interrotta e la pipe si ferma (c'è sempre un po' di stallo), vengono eseguite le istruzioni ibernata a partire dal caricamento di R1. A questo punto R1 punta a 7 quindi in 7 viene scritto il valore caricato e le operazioni sospese scendono a 1 e viene impostato 0 nel bit che ne indica la validità.

	operando1		operando2		Contenitore
	tag	valore	tag	valore	
ADD					
ADD	8		16		15
MUL					
MUL					
DIV					
BOO					
LOAD			MEMA		7
STORE					

Al realizzarsi del caricamento, si sblocca anche R2, che viene ad essere calcolato come $R1+R3$, viene ridotto il contatore delle operazioni sospese su OPR(8) che scende a 1 e viene cancellata dalla tabella la riga rispettiva a questa operazione che ormai è stata eseguita.

Si passa all'istruzione i+2, viene risolto anche R4, il numero di operazioni in attesa su 8 viene posto a 0, il valore contenuto nel bit occupato pure viene variato dal valore che indica la validità del contenuto e OPR(8) può essere riutilizzato, in quanto non c'è nessuna istruzione che attende un valore contenuto in OPR(8) e nessun registro che punta a tale valore della tabella, questo non succede a OPR(7), in quanto seppure il contatore delle operazioni in attesa di quel valore può azzerarsi, il bit di occupato rimane, perché c'è R1 che punta ancora a 7.

In pratica negli 8 registri c'è ancora un puntatore ad OPR(7).

Il punto chiave è che ci sono due controlli separati che tengono in vita un contenitore. Per poter cancellare e riciclare un OPR, *entrambi* i lucchetti devono essere aperti:

Lucchetto 1: Il Contatore di Attesa

Lucchetto 2: Il Puntatore Architeturale (Bit di Occupato): Il suo contatore scende a 0, MA il lucchetto architeturale resta chiuso (R1 punta ancora lì).

Nella fase di decodifica viene ad essere riempita la tabella di ibernazione e aggiornati i valori dei puntatori contenuti nei registri, *nella fase di esecuzione* le tabelle vengono

variate, lavorando sulla tabella dei valori effettivi puntati dai registri, quindi in EX si modifica e si cancellano valori delle tabelle.

Si capisce che il modello di Von Neumann non è rispettato, la cosa risulta ancora più chiara in occasione di una interruzione causata per esempio dall'overflow dell'istruzione $i+3$ ($R2=R6+R7$), perché in tal caso si interromperebbe prima dell'esecuzione di istruzioni precedenti, credendo di interrompere avendo una macchina modificata dall'esecuzione di tali istruzioni ibernate.

Esempio 2:

i-1 **R1 = MEMA;**
i **R2 = R1 x MEMB;**
i+1 **R3 = R2 / MEMC;**
i+2 **R4 = R4 + R3;**

R0		OPR(0)			
R1	4	OPR(1)	17	0	0
R2		OPR(2)	14	0	1
R3		OPR(3)			
R4	1	OPR(4)		1	0
R5		OPR(5)			
R6		...			
R7		OPR(N)			

La prima istruzione prevede una lettura in memoria che però va in cache miss. Quindi bisogna inserirlo nella tabella di ibernazione nella sezione LOAD. Come contenitore non possiamo mettere né 1 né 2 perché erano già occupati. Mettiamo 4. Quindi nei registri il puntatore cambia e in R1 mettiamo 4 quindi punta ad OPR(4).

Siccome R1 verrà sovrascritto, il vecchio valore non serve più e allora OPR(2) viene liberato mettendo a 1 il *bit occupato*.

L'istruzione quindi aspetta un valore che dovrà essere caricato in OPR(4) quindi il contatore passa da 0 a 1.

La seconda istruzione $R2 = R1 \times \text{MEMB}$ non può essere subito eseguita, siccome il valore di R1, contenuto in OPR(4), non è ancora disponibile. Essendoci nella tabella di ibernazione, una stazione libera per la moltiplicazione, l'istruzione è ibernata. Il campo 'primo operando' conterrà 4 (OPR(4)), il campo 'secondo operando' MEMB. Il risultato è il puntatore ad un registro operando libero per R2, supponiamo ad esempio sia OPR(3).

Quindi OPR(4) ha due istruzioni sospese, e OPR(3) ne ha 1.

La terza istruzione $R3 = R2 / \text{MEMC}$ non può essere eseguita, perciò è ibernata nella stazione della divisione, inserendo OPR(3) come primo operando e MEMC come secondo. Il risultato sarà memorizzato, ad esempio, in OPR(5). Quindi OPR(3) sale a 2 istruzioni sospese, OPR(5) da 0 passa a 1.

Per chiudere la propria operazione, in questo momento dell'elaborazione OPR(3) ha bisogno di OPR(4) e OPR(5) ha a sua volta bisogno di OPR(3).

Per l'ultima istruzione $R4 = R4 \times R3$ supponiamo che R4 punta ad OPR(1), che contenente il vecchio valore di tale registro. Il risultato può essere messo in OPR(0), e dovrà attendere il caricamento di OPR(5). OPR(1) contiene un vecchio valore per R4 e quindi, dopo che sarà stata compiuta l'operazione in lettura tenuta in sospeso su di esso, sarà liberato. Questo ci consente quindi di usare il vecchio valore di R4 perché nella riga della tabella di ibernazione mettiamo i puntatori agli operandi.

R0		OPR(0)		1	0
R1	4	OPR(1)	17	1	0
R2	3	OPR(2)	14	0	1
R3	5	OPR(3)		2	0
R4	0	OPR(4)		2	0
R5		OPR(5)		2	0
R6		• • •			
R7		OPR(N)			

Conflitti nei sistemi basati su pipelining – Gestione delle ISR in sistemi pipelined

Se non ho le interruzioni precise, se ho l'interruzione da parte di un'istruzione non è detto che quelle ibernate non mi avrebbero interrotto.

Mi devo chiedere se mi potevano interrompere anche quelle che avevo prima. Posso capire il grado di decisione. Mi ha interrotto una riga e faccio l'abort però direi anche stai attento perché non è detto che quella è l'unica, ma ti potevano interrompere anche quelle di prima.

Le interruzioni esterne sono le più facili da gestire. Supponiamo di avere una pipe nella quale, in un certo momento, sono state attivate un certo numero di istruzioni. In base al modello classico di Von Neumann, occorre terminare l'operazione in corso nella pipe (e le eventuali operazioni ibernate) e salvare lo stato del processore, prima di servire una interruzione. Seguendo tale modello, quindi, quando giunge un'interruzione da parte di una periferica di I/O, la procedura di servizio dell'interruzione sarà inserita nella pipe come una qualsiasi altra istruzione.

Più critico è il caso **delle interruzioni interne**. Poiché si tratta di interruzioni che hanno effetto all'interno della pipe e non all'esterno, tali eventi possono dare luogo ad un comportamento del programma che è diverso da quello di Von Neumann. Può accadere

che, in una sequenza i_1 - i_2 , l'istruzione i_2 (che viene per seconda) generi un'eccezione (ad es. la divisione per zero, che blocca il programma) prima che l'istruzione i_1 abbia avuto il tempo di essere eseguita.

Si può comprendere una **differenza sostanziale tra processori CISC e processori RISC**. Abbiamo visto che i RISC sono caratterizzati da una minore complessità, codici operativi più semplici ed in numero inferiore, operandi solo registro, ma alla fine sostanzialmente lo stesso numero di transistor di un processore CISC, ciò è dovuto al fatto che nei RISC dove si risparmia silicio sulla complessità della rete di controllo, si investe in registri e tabelle proprio per migliorare le prestazioni, cercando di non fermare mai la pipe, ma anche a causa del fatto che nei RISC non c'è il concetto di rete di controllo unitaria ma abbiamo una netta divisione in fasi, proprio per cercare di ridurre al minimo le influenze tra fasi.

Con l'introduzione del sistema a pipeline e del meccanismo dell'internal forwarding, che altera la sequenzialità delle istruzioni eseguite dal processore, nasce il problema della gestione delle interruzioni. Abbiamo 2 soluzioni:

- **Alcuni processori rinunciano del tutto ad avere interruzioni precise**: vige una specie di legge del più forte, in cui conta l'interruzione che arriva per prima. Il comportamento sequenziale di Von Neumann quindi non è assicurato. Le istruzioni sono out order, cioè non sono eseguite necessariamente in ordine (come avviene nei processori che implementano l'internal forwarding) e di conseguenza generano interruzioni non precise. Così facendo, si demanda al Software di gestione il compito di stabilire se oltre alla istruzione che ha generato l'eccezione possano essercene altre "non corrette" tra quelle presenti in pipe e non ancora completate all'atto della interruzione. Questa, indubbiamente è una soluzione poco efficiente, ma da non sottovalutare se si considera che la percentuale di istruzioni che generano eccezioni in un programma è molto bassa.
- Altri processori nei quali pure le istruzioni sono out order riescono ciononostante a ricostruire delle **interruzioni precise**. Le relative tecniche sono al solito di tipo conservativo oppure ottimistico.
 - Nel **caso conservativo**, il processore fa procedere nella pipe una istruzione finché è sicuro che non possa essere fonte di interruzione, e solo a questo punto ne avvia un'altra. Il parallelismo interno della pipe è dunque limitato, a vantaggio della gestione precisa delle eccezioni.
 - Nel **caso ottimistico**, il processore va avanti comunque, e se effettua elaborazioni errate deve poter tornare indietro (roll-back), cosa che può fare solo se è in possesso dell'immagine dello stato del sistema. Le tecniche che consentono di tornare indietro sono fondamentali nei sistemi transazionali. Tecniche che consentono di annullare delle operazioni.

Per poter effettuare il rollback esistono due tecniche:

- tecnica dell'history buffer
- tecnica del check pointing

Tecnica del check pointing

Consiste nell'effettuare delle fotografie dello stato dei registri del processore (program counter incluso) e nel caso in cui una cosa non funziona torna alla fotografia precedente, ovvero si copia il contenuto della fotografia dell'ultimo check point nei registri, facendo riprendere l'esecuzione del programma. Non è molto vantaggioso perché queste fotografie si devono conservare. Occupano uno spazio di memoria.

i R4 = R1;

i+1 R2 = R0+ R6;

i+2 R3 = R1/R5

Quando arrivo ad un'istruzione e ho un'interruzione posso decidere di ripristinare il sistema impostandolo in modo sequenziale, ma solo se ho fatto una fotografia alla prima istruzione ibernata.

Il problema nasce nella gestione della memoria in presenza di istruzioni che ne alterano il contenuto e che non si sarebbero dovute eseguire perché successive all'istruzione che ha generato l'eccezione. Mettiamo caso che un'istruzione è stata ibernata e quelle dopo vengono eseguite poiché hanno a disposizione tutti gli operandi. Quando l'istruzione ibernata si sblocca verrà eseguita e mettiamo caso lancia un'eccezione. A questo punto si prenderebbe un check Point e si ricopiarebbero i valori nei registri ritornando ad uno stato precedente.

La difficoltà dell'approccio consiste nel sapere in quali momenti devono essere salvati gli stati sicuri; non è possibile farlo spesso per non sovraccaricare il sistema, ma farlo raramente causa un ritardo maggiore quando è necessario rifare le operazioni. Occorre, quindi, trovare un giusto compromesso fra le due esigenze.

Se lo facciamo in hardware siamo sicuri, perché l'interruzione la sospendiamo, ritorniamo sopra e facciamo tutte le cose sopra, quindi non diamo corso all'interruzione. Quando non si può fare?

Quando porto un dato dal registro alla memoria. Quindi dovrei ricordare anche pezzi di memoria esterni. La cosa importante è capire se riesco a ripristinare lo stato.

History buffer

Questa tecnica consiste nell'utilizzare un'area di memoria in cui conservare le istruzioni eseguite e i valori degli operandi. Ogni riga verrà cancellata solo se tutte quelle che la precedono si sono concluse senza generare interruzioni.

Definiamo ed eseguiamo delle operazioni di UNDO in cui andiamo a disfarcì delle operazioni successive a quelle che ha generato l'interruzione.

i MEMA = R1;

i+1 R2 = R4+ R6;

i+2 R3 = R1/R5

l'istruzione 2 genererà un'interruzione, mentre i potrebbe generarla. L'istruzione i inoltre impiega più tempo di i+1 e i+2.

Quando la $i+1$ termina viene memorizzato nel registro R2 il valore, mentre il vecchio valore è memorizzato nel HistoryBuffer. Successivamente la $i+2$ genera un'interruzione e nell'HB è memorizzata una proposta di interruzione da parte di $i+2$ -

i	
i+1	14
i+2	INT

Come vediamo nell'HB all'istruzione i ancora non è associato nulla perché l'istruzione non è terminata.

All'istruzione $i+1$ è associato il vecchio valore e all'istruzione $i+2$ è associato INT.

Ora bisogna vedere se le istruzioni precedenti sono terminate tutte quante.

L'istruzione i ancora non è terminata quindi abbiamo due strade:

- l'istruzione i non crea eccezione e quindi lo stato delle istruzioni precedenti la riga $i+2$ è valido. A questo punto vengono eliminate perché la regola è che una riga venga eliminata se tutte le precedenti sono terminate senza causare interruzioni. Quindi rimane solo la riga $i+2$ che viene servita.
- Se invece la riga i lancia un'eccezione lo stato precedente ad $i+2$ non è valido. Quindi bisogna fare UNDO delle istruzioni $i+1$ e $i+2$. Tale operazione coincide con il riscrivere 14 nel registro R2 e annullare l'istruzione $i+2$. Quindi verrà servita l'istruzione i .

La grandezza del HB deve essere pari al numero di istruzioni che possono generare contemporaneamente interruzione.

La gestione del HB se non ci fossero interruzioni non costerebbe nulla, sarebbe gestito totalmente in hardware e le operazioni di copia vengono fatte in parallelo alle normali operazioni.

Il taglio del Log è detto **SNAP SHOT**. Nei sistemi operativi memorizzare ogni stato produrrebbe un file Log troppo lungo, perciò si memorizzano solo alcuni punti detti checkpoint.

Entrambe le tecniche sono attuabili soltanto solo quando le istruzioni su cui operano sono invarianti, ovvero non alterano lo stato del sistema (non scrivono in memoria).

Ad esempio per un sistema Bancomat non è possibile prevedere una gestione delle interruzioni con la tecnica del roll-back, perché alcune operazioni come quella di prelievo contante non sono invarianti.

Inoltre bisogna assolutamente evitare che un'istruzione che va a scrivere in memoria, venga interrotta da un'istruzione che la segue. Questa cosa non può avvenire perché la maggior parte delle operazioni generano eccezioni nelle fasi di ID oppure EX.

Debugging

In linea di massima le interruzioni precise non servono, perché molti processori quando sono in questa modalità fanno in modo da non attivare la pipe; così, se si verifica una eccezione durante il debugging si può sempre sapere con certezza quale istruzione l'ha causata (informazione impossibile da reperire se la pipe è attiva). Tuttavia in contesti real-time o nei quali è comunque richiesta un'alta velocità, occorre far funzionare il processore nelle sue condizioni reali di funzionamento e quindi non si può non attivare la pipe mentre è in esecuzione il debugger.